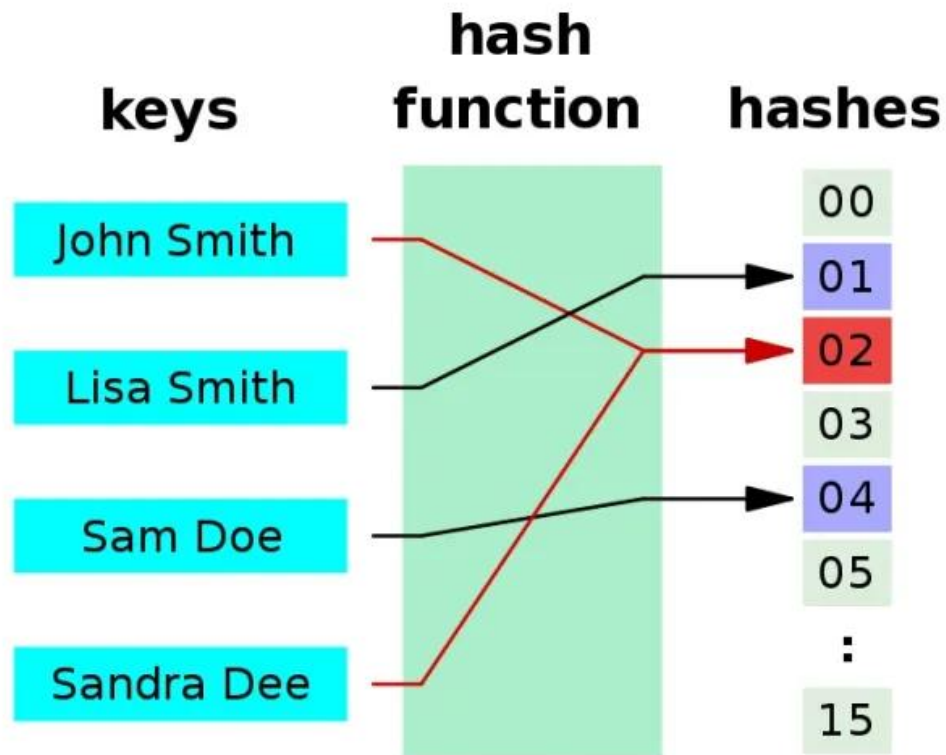


Hashing



Hashing is a technique that is used to **uniquely identify a specific object from a group of similar objects**

Assume that you have an object and you want to assign a key to it to make searching easy.

To store the **key/value** pair, you can use a simple array like a data structure where keys (integers) can be used directly as an index to store values.

However, in cases where the keys are large and cannot be used directly as an index, you should use ***hashing***.

- In hashing, large keys are converted into small keys by using **hash functions**.
- The values are then stored in a data structure called **hash table**.
- The idea of hashing is to distribute entries (key/value pairs) uniformly across an array.

INDEXING VERSUS HASHING

The **main difference** between indexing and hashing is that

the **indexing optimizes the performance of a database by reducing the number of disk accesses to process queries**

while hashing calculates the direct location of a data record on the disk without using index structure.

INDEXING	HASHING
A data structure technique to efficiently retrieve records from the database files based on some attributes on which the indexing has been done	An effective technique to calculate the direct location of a data record on the disk without using index structure
Uses data reference that holds the address of the disk block with the value corresponding to the key	Uses mathematical functions called hash functions to calculate direct locations of data records on the disk
Does not work well for large databases	Works well for large databases
	Visit www.PEDIAA.com

Hashing is implemented in two steps:

- An element is converted into an integer by using a hash function.
- ~~This element~~ it can then be used as an index to store the original element, which falls into the hash table.

The element is stored in the hash table where it can be quickly retrieved using hashed key.

```
hash = hashfunc(key)  
index = hash % array_size
```

In this method, the hash is independent of the array size and it is then reduced to an index (a number between 0 and array_size – 1) by using the modulo operator (%).

Hash function

A hash function is any function that can be **used to map a data set of an arbitrary size to a data set of a fixed size**, which falls into the hash table. The values returned by a hash function are called hash values, hash codes, hash sums, or simply hashes.

Good hash function has following basic requirements:

Easy to compute: It should be easy to compute and must not become an algorithm in itself.

Uniform distribution: It should provide a uniform distribution across the hash table and should not result in clustering.

Less collisions: Collisions occur when pairs of elements are mapped to the same hash value, collision should be avoided.

Note: Irrespective of how good a hash function is, collisions are bound to occur.

Hash table

A hash table is a data structure that is used to store keys/value pairs.

- ✓ It uses a hash function to compute an index into an array in which an element will be inserted or searched.
- ✓ Hashing work well by using a good hash function
- ✓ Under reasonable assumptions, the average time required to search for an element in a hash table is **$O(1)$** .

Division method

In this method,

key k to be mapped into one of the m slots in the hash table.

- The key is divided by m and the remainder of the division is taken as index of the hash table

The hash function is defined as

$$h(k) = k \bmod m$$

Example

Consider hash table with 9 slots. To which slot key=132 will map to ?

Using $h(k) = k \bmod m$

$$h(132) = 132 \bmod 9 = 6$$

Note – In division method, certain values of m should be avoided, preferable values for m are prime numbers and not too close to exact powers of 2

Multiplication Method

This method operates in two steps .

- **Step1**- the key value is multiplied by constant A in the range $0 < A < 1$ and abstract the fractional part of value kA
- **Step2**- the fractional value obtained is then multiplied by m and the floor of the result is taken as the hash value

$$\text{Thus} \quad h(k) = \text{floor} (m (kA \bmod 1))$$

(Knuth suggested the efficient value for A as **0.6180339887**)

Example – Consider a hash table with 1000 slots, then key= 123456 will map to which slot ?

$$h(123456) = \text{floor} (1000 (123456 \times 0.61803 \bmod 1)) = \text{floor} (41.151) = 41$$

$$(123456 \times 0.61803 = 76300.0041151)$$

Example – Consider a hash table with 300 slots, then key= 3456 will map to which slot ?

Folding Method

It is performed in two step

- **Step1-** the key value is divided into number of parts say $k_1, k_2, k_3, \dots, k_r$, where each part has same number of digits except the last part (which can have lesser digits)
- **Step2-** these parts are added together and the hash value is obtained by ignoring the last carry.
- Ex – Consider a hash table with 100 slots and key values are 9235, 714, 71458

k	9235	714	71458
Parts	92 , 35	71, 4	
Sum	127	75	
$h(k)$	27	75	

Folding Method

It is performed in two step

- **Step1-** the key value is divided into number of parts say $k_1, k_2, k_3, \dots, k_r$, where each part has same number of digits except the last part (which can have lesser digits)
- **Step2-** these parts are added together and the hash value is obtained by ignoring the last carry.
- Ex – Consider a hash table with 100 slots and key values are 9235, 714, 71458

k	9235	714	71458
Parts	92 , 35	71, 4	71, 45, 8
Sum	127	75	124
$h(k)$	27	75	24

Need for a good hash function

Let us understand the need for a good hash function. Assume that we have to store strings in the hash table by using the hashing technique

{“abcdef”, “bcdefa”, “cdefab” , “defabc” }.

To compute the index for storing the strings,

We use a hash function that states the following:

The index for a specific string will be equal to the sum of the ASCII values of the characters modulo 599.

Index				
0				
1				
2	abcdef	bcdefa	cdefab	defabc
3				
4				
-				
-				
-				
-				

**The ASCII values of
a, b, c, d, e, and f are
97, 98, 99, 100, 101, and 102 respectively**

The hash function will **compute the same index** for all the strings and the strings will be stored in the hash table

Proposing a good Hash Function

Let's try a different hash function.

The index for a specific string will be equal to sum of ASCII values of characters multiplied by their respective order in the string after which it is modulo with 2069 (prime number).

String	Hash function	Index
abcdef	$(97*1 + 98*2 + 99*3 + 100*4 + 101*5 + 102*6)\%2069$	38
bcdefa	$(98*1 + 99*2 + 100*3 + 101*4 + 102*5 + 97*6)\%2069$	23
cdefab	$(99*1 + 100*2 + 101*3 + 102*4 + 97*5 + 98*6)\%2069$	14
defabc	$(100*1 + 101*2 + 102*3 + 97*4 + 98*5 + 99*6)\%2069$	11

The ASCII values of
a, b, c, d, e, and f are
97, 98, 99, 100, 101, and 102 respectively

Index	
0	
1	
-	
-	
-	
11	defabc
12	
13	
14	cdefab
-	
-	
-	
-	
23	bcdefa
-	
-	
-	
38	abcdef
-	
-	

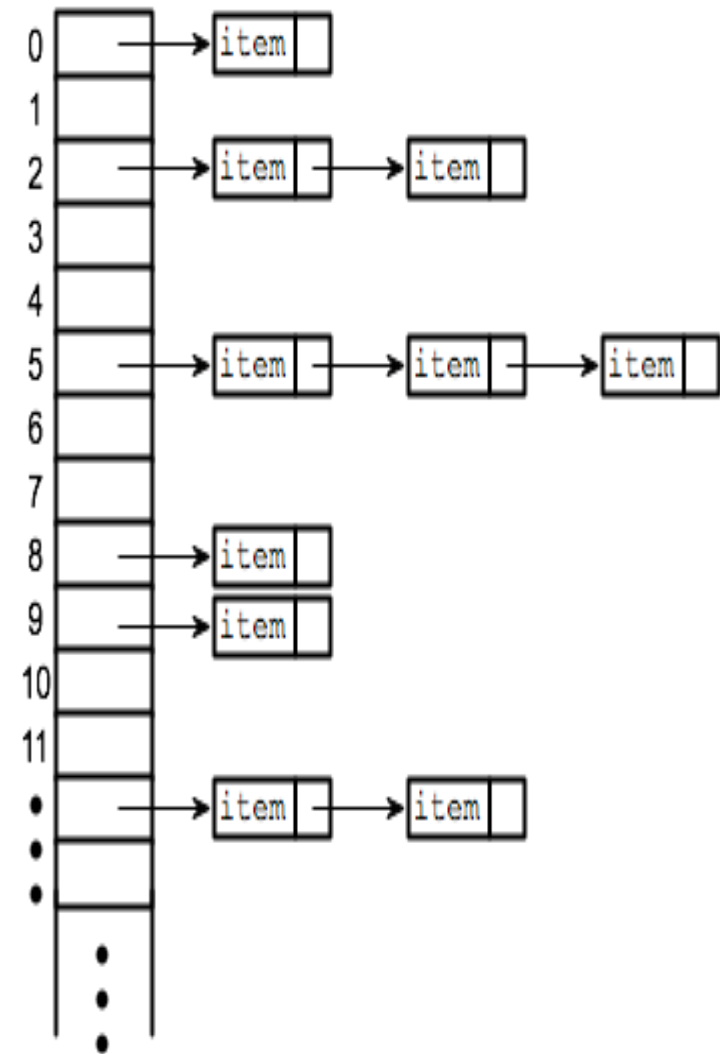
Collision resolution techniques

Separate chaining (open hashing)

It is one of the most commonly used collision resolution techniques. It is usually implemented using linked lists.

In separate chaining, **each element of the hash table is a linked list.**

- ❖ To store an element in the hash table we insert it into a specific linked list.
- ❖ If there is any collision (i.e. two different elements have same hash value) then store both the elements in the same linked list.



Ex Separate chaining (open hashing)

Consider the insertion of elements

5, 28, 19, 15, 20, 33, 12, 17, 10

into a chained-hash-table with **9 slots**. The table uses the following hash function

$$h(k) = k \bmod 9$$

Soln — elements are inserted as

$$h(5) = 5 \bmod 9 = 5$$

$$h(28) = 28 \bmod 9 = 1$$

$$h(19) = 19 \bmod 9 = 1$$

$$h(15) = 15 \bmod 9 = 6$$

$$h(20) = 20 \bmod 9 = 2$$

$$h(33) = 33 \bmod 9 = 6$$

$$h(12) = 12 \bmod 9 = 3$$

$$h(17) = 17 \bmod 9 = 8$$

0	X
1	X
2	X
3	X
4	X
5	X
6	X
7	X
8	X

Elements are inserted in beginning to keep $O(1)$

insert elements using Separate chaining (*open hashing*)

15, 28, 29, 15, 20,

into a chained-hash-table with 6 **slots**.

The table uses the following hash function

$$h(k) = k \bmod 6$$

Soln – elements are inserted as

0	X
1	X
2	X
3	X
4	X
5	X

Linear probing (open addressing or closed hashing)

Probing – the process of examining the slots in the hash table

"open addressing" refers to the fact that the location or address of the item is not determined by its hash value.

In **open addressing**, instead of in linked lists, all entry **records are stored in the array itself**.

When a new entry has to be inserted

- the hash index of the hashed value is computed and then the array is examined (starting with the hashed index).
- If the slot at the **hashed index is unoccupied**, then the entry record is inserted in slot at the hashed index
- else it proceeds in some probe sequence until it finds an unoccupied slot.

Probe sequence is the sequence that is followed while traversing through entries.

In different probe sequences, we have different intervals between successive entry slots or probes.

When searching for an entry

- the array is scanned in the same sequence until either the target element is found or an unused slot is found.
- unused slot indicates that there is no such key in the table.

In Linear probing the interval between successive probes is fixed (usually to 1).

Ex linear probing uses the following hash function

$$h(k, i) = [h'(k) + i] \bmod m$$

Here m is the size of the hash table

$h'(k) = k \bmod m$ (hash fns)

i will range from 0 to $m-1$

Alternatively

Let's assume that the hashed index for a particular entry is **index**.

The probing sequence for linear probing will be:

$\text{index} = \text{index} \% \text{hashTableSize}$

$\text{index} = (\text{index} + 1) \% \text{hashTableSize}$

$\text{index} = (\text{index} + 2) \% \text{hashTableSize}$

$\text{index} = (\text{index} + 3) \% \text{hashTableSize}$

Consider inserting the keys **76, 26, 37, 59** into hash table of **size m =11** using liner probing .

Use hash function $h'(k) = k \bmod m$

Solution

using $h(k,i) = [h'(k) + i] \bmod m$

Key k =76

$h(76,0) = (76 \bmod 11 + 0) \bmod 11 = (10 + 0) \bmod 11 = 10 \bmod 11 = 10$ (empty)

Key k = 26

$h(26,0) = (26 \bmod 11 + 0) \bmod 11 = (4 + 0) \bmod 11 = 4 \bmod 11 = 4$ (empty)

Key k = 37

$h(37,0) = (37 \bmod 11 + 0) \bmod 11 = (4 + 0) \bmod 11 = 4 \bmod 11 = 4$ (occupied)

Finding next probe sequence

$h(37,1) = (37 \bmod 11 + 1) \bmod 11 = (4 + 1) \bmod 11 = 5 \bmod 11 = 5$ (empty)

Key k = 59

$h(59,0) = (59 \bmod 11 + 0) \bmod 11 = (4 + 0) \bmod 11 = 4 \bmod 11 = 4$ (occupied)

Finding next probe sequence

$h(59,1) = (59 \bmod 11 + 1) \bmod 11 = (4 + 1) \bmod 11 = 5 \bmod 11 = 5$ (occupied)

Finding next probe sequence

$h(59,2) = (59 \bmod 11 + 2) \bmod 11 = (4 + 2) \bmod 11 = 6 \bmod 11 = 6$ (empty)

Quadratic Probing

Linear probing is easy to implement but it suffers from primary clustering

Cluster means block of occupied slots

Primary clustering refers many such blocks separated by free slots

It increases number of probes to find a free slot

Quadratic probing is similar to linear probing and the only difference is the interval between successive probes or entry slots.

Quadratic Probe uses the following hash function

$$h(k, i) = [h'(k) + c_1 i + c_2 i^2] \bmod m$$

Here m is the size of the hash table

$h'(k) = k \bmod m$ (hash fns)

c_1 and c_2 are constants not equal 0

i is probe number

In Quadratic probing

- when the slot at a hashed index for an entry record is already occupied, we start traversing until you find an unoccupied slot.
- The interval between slots is computed by adding the successive value of an arbitrary polynomial in the original hashed index.
- Let us assume that the hashed index for an entry is **index** and at **index** there is an occupied slot.

The probe sequence will be as follows:

- $\text{index} = \text{index} \% \text{hashTableSize}$
 $\text{index} = (\text{index} + 1^2) \% \text{hashTableSize}$
 $\text{index} = (\text{index} + 2^2) \% \text{hashTableSize}$
 $\text{index} = (\text{index} + 3^2) \% \text{hashTableSize}$
- and so on...

Consider inserting the keys **76, 26, 37, 59** into hash table of **size m =11** using quadratic probing . [given $c_1= 1$ and $c_2=3$]

Use hash function **$h'(k) = k \bmod m$**

Solution

using **$h(k, i) = [h'(k) + c_1i + c_2i^2] \bmod m = [k \bmod 11 + 1 \times i + 3 \times i^2] \bmod 11$**

Key k =76

$h(76,0) = [76 \bmod 11 + 0 + 3 \times 0] \bmod 11 = (10 + 0 + 0) \bmod 11 = 10 \bmod 11 = 10$ (empty)

Key k = 26

$h(26,0) = [26 \bmod 11 + 0 + 3 \times 0] \bmod 11 = (4 + 0 + 0) \bmod 11 = 4 \bmod 11 = 4$ (empty)

Key k = 37

$h(37,0) = [37 \bmod 11 + 0 + 3 \times 0] \bmod 11 = (4 + 0 + 0) \bmod 11 = 4 \bmod 11 = 4$ (occupied)

Finding next probe sequence

$h(37,1) = [37 \bmod 11 + 1 + 3 \times 1] \bmod 11 = (4 + 1 + 3) \bmod 11 = 8 \bmod 11 = 8$ (empty)

Key k = 59

$h(59,0) = [59 \bmod 11 + 0 + 3 \times 0] \bmod 11 = (4 + 0 + 0) \bmod 11 = 4 \bmod 11 = 4$ (occupied)

Finding next probe sequence

$h(59,1) = [59 \bmod 11 + 1 + 3 \times 1] \bmod 11 = (4 + 1 + 3) \bmod 11 = 8 \bmod 11 = 8$ (occupied)

Finding next probe sequence

$h(59,2) = [59 \bmod 11 + 2 + 3 \times 4] \bmod 11 = (4 + 2 + 12) \bmod 11 = 18 \bmod 11 = 7$ (empty)

Double hashing

Double hashing is similar to linear probing and the only difference is the interval between successive probes.

Here, the interval between probes is computed by using two hash functions.

Lets linear probing uses the following hash function

$$h(k,i) = [h_1(k) + i.h_2(k)] \bmod m$$

Here m is the size of the hash table

$h_1(k) = k \bmod m$ (hash fns)

$h_2(k) = k \bmod m'$ [hash fns with slightly less m (say $m-1$)]

i is 0 to $m-1$

Consider inserting the keys **76, 26, 37, 59** into hash table of **size m =11** using double hashing.

Use hash function $h_1(k) = k \bmod 11$ and $h_2(k) = k \bmod 9$

Solution

using $h(k,i) = [h_1(k) + i.h_2(k)] \bmod m$

Key k =76

$h_1(76) = (76 \bmod 11) = 10$ and $h_2(76) = 76 \bmod 9 = 4$

$h(76,0) = (10 + 0 \times 4) \bmod 11 = 10 \bmod 11 = 10$ (empty)

Key k = 26

$h_1(26) = (26 \bmod 11) = 4$ and $h_2(26) = 26 \bmod 9 = 8$

$h(26,0) = (4 + 0 \times 8) \bmod 11 = 4 \bmod 11 = 4$ (empty)

Key k = 37

$h_1(37) = (37 \bmod 11) = 4$ and $h_2(37) = 37 \bmod 9 = 1$

$h(37,0) = (4 + 0 \times 1) \bmod 11 = 4 \bmod 11 = 4$ (occupied)

Finding next probe sequence

$h(37,1) = (4 + 1 \times 1) \bmod 11 = 5 \bmod 11 = 5$ (empty)

Consider inserting the keys **76, 26, 37, 59** into hash table of **size m =11** using double hashing.

Use hash function **$h_1(k) = k \bmod 11$** and **$h_2(k) = k \bmod 9$**

Solution

using **$h(k,i) = [h_1(k) + i.h_2(k)] \bmod m$**

Key k =76

$h_1(76) = (76 \bmod 11) = 10$ and **$h_2(76) = 76 \bmod 9 = 4$**

$h(76,0) = (10 + 0 \times 4) \bmod 11 = 10 \bmod 11 = 10$ (empty)